Journal of Biosystems Engineering
https://doi.org/10.1007/s42853-022-00166-6

Online ISSN 2234-1862
Print ISSN 1738-1266

ORIGINAL ARTICLE

Dynamic Beehive Detection and Tracking System Based on YOLO V5 and Unmanned Aerial Vehicle

Peng Gao[1] · Kangbeen Lee[2,3] · Lukas Wiku Kuswidiyanto[2,3] · Seung-Hwa Yu[4] · Kai Hu[1] · Gaotian Liang[1] · Yufeng Chen[1] · Weixing Wang[1] · Fei Liao[1] · Yu Seok Jeong[4] · Moon-Seok Jeon[4] · Inchan Choi[4] · Xiongzhe Han[2,3]

Received: 28 September 2022 / Revised: 26 October 2022 / Accepted: 27 October 2022
© The Author(s), under exclusive licence to The Korean Society for Agricultural Machinery 2022

Abstract

Purpose With urban development and improvements in human living conditions, wild beehives in densely populated areas present a threat to human safety. The traditional manual method of clear beehives may result in secondary injury to humans.
Methods This paper proposes a beehive detection model based on YOLO V5 by introducing the Shuffle Block V2 and depthwise separable convolution (DSC) modules to decrease the original model parameters. The model can be deployed on edge computation devices such as Raspberry 4B with good detection accuracy. The PID algorithm and dual servo motors were combined with the object detection model to track the beehive automatically. The results of experiments showed that the inference speed of the improved beehive detection model was 92.5% faster than the original YOLO V5s model, although the detection accuracy and other indicators were not significantly different.
Results The accuracy of the system in this study was as high as 96% in real-time detection, and the maximum recognition distance was 2.5 m. The performance test results of the system deployed on an unmanned aerial vehicle (UAV) showed that 90% of the beehive tracking process could be completed within 2 s, positioning the object in the center of the images collected by the camera. At the same time, when the UAV was moving at random, the detection and tracking system could still follow the beehive quickly and automatically.
Conclusion The detection model and tracking system established in this paper provide important support to reduce the secondary damage to the rescue workers that may occur in beehive governance.

Keywords Automatic control · Beehive · Edge computing · Object detection · YOLO

Introduction

Bees, wasps, and other insects have essential ecological and economic value to the earth (Brugnerotto et al., 2021; Nader et al., 2021). However, with increasing urbanization worldwide, wild hives in densely populated urban areas may cause unexpected harm to humans. Part of wasps would harm humans because of their neurotoxins. It is common in China and other Asia countries for the residents to call the government for help over beehive presence (Yu, 2021). It is necessary to take measures to destroy beehives in time to protect residents from damage. At present, traditional beehive disposal methods mainly rely on fire rescue professionals and other officials, which would be harm rescue workers when doing tasks related to beehive governance. Using flamethrowers to destroy beehives is effective, although it also poses potential hazards such as fire and death. Therefore, developing a new method to process beehives is necessary to reduce the potential risk to personnel and improve governance efficiency.

The unmanned aerial vehicle (UAV) has been widely used in agriculture, fire prevention and disaster relief, air quality monitoring, and other fields (Rahman et al., 2021). Wu et al. (2021) developed a system based on a UAV to monitor the safety dip angle of a foundation pit in urban development and construction. The research proved the potential benefits of UAVs in safety management. Dainelli et al. (2021) reviewed the latest remote sensing technology for UAVs. The UAV serves as a good platform for the fast hyperspectral monitoring of extensive forests. Thus, the UAV offers benefits, such as autonomous cruise and remote control, suitable for rapid unmanned tasks. Therefore, it would be

✉ Xiongzhe Han
hanxiongzhe@kangwon.ac.kr

Extended author information available on the last page of the article

Published online: 14 November 2022

effective to use the UAV in urban beehive management to improve the efficiency of beehive governance and reduce casualties. On the other hand, UAVs and flamethrowers are applied to destroy beehives commonly in China. However, a UAV and flamethrower are generally used with a camera to focus on the beehive and destroy it manually. And the angle of the flamethrower is also controlled manually. The manual method cannot aim at the beehive accurately. The deviation of the tracking object could cause serious consequences. Therefore, it is necessary to develop an automatic object detection and focusing method that improves the efficiency significantly in destroying the beehives.

The key to tracking beehives automatically is accurate detection, and deep learning is the most common object detection method. Gunes and Gungormus (2022) used the region-based convolutional neural networks (R-CNN) model to detect bee larvae with a camera to capture images. After the R-CNN model was trained, the bees were recognized accurately. Zhuang et al. (2022) used a Single Shot Multi-Box Detector (SSD) model for the automatic detection and count of bees. This study showed that the SSD model offered good accuracy and fast detection speed. The model also recognized bees effectively under complex environmental conditions. However, the SSD model required the region of interest (ROI) to classify the region further to realize the objects. Therefore, SSD achieved good accuracy but also required considerable computing resources.

You only look once (YOLO) is a single-stage object detection model based on the detected boundary box, transferring the object detection into regression analysis, simplifying the training process (Huang et al., 2018). Although its accuracy is reduced, the object detection speed could be significantly improved, especially on low computation devices. Wang and Liu (2021) proposed an improved YOLO model by adding dense network modules, which achieved the high-precision detection of small objects such as tomatoes, and the model reasoning speed was also increased. However, the original YOLO model still requires considerable computing resources to achieve the expected accuracy and speed of object detection. With the development of edge computing devices and model deployment, the miniaturization and lighter weight of deep learning models have become a critical application trend. When the object detection model is deployed on mobile devices or a UAV, the miniaturization of the model directly determines the object detection result (Tian et al., 2019).

Wahyutama and Hwang (2022) implemented the YOLO V4 model on a Raspberry Pi to classify and recognize trash. The test results showed that Raspberry Pi could reach two frames per second (FPS) with an input image source of 320×320 resolution. The entire YOLO V4 model was used in the research; therefore, it was necessary to simplify the original YOLO V4 to achieve better object detection performance. On the other hand, considering the detection object of this study, the detection accuracy may also change when the distance changes between the camera and the tomatoes. Legaspi et al. (2021) used the YOLO V3 model to conduct the real-time detection of small whiteflies and fruit flies. The model was also implemented on Raspberry Pi with 83.07% accuracy on object detection, whereas the simplified model was not used, limiting the performance on edge computing nodes.

The main contributions of the beehive detection and tracking system proposed in this paper are as follows:

(1) A simplified YOLO V5 model was established to improve object detection performance for beehives based on the Raspberry Pi 4B edge device.
(2) According to the results of the object detection model, the object centroid and the offset for the image center point were dynamically calculated to complete the closed-loop control of the automatic beehive tracking system.
(3) The performance experiments for the object detection model and tracking system were conducted to provide a practical, theoretical reference and a data reference for the efficient governance of urban beehives.

Materials and Methods

Description of Hardware System

The entire compositions for dynamic beehive detection and automatic tracking are shown in Fig. 1. The UAV carried the control box during the experiment. The beehive was detected using a built-in Raspberry Pi 4B with a 500-megapixel camera. The offsets of the object were calculated to be used as tracking parameters to control the two motion motors independently to perform automatic tracking of the beehive. The control box was adjusted continuously by the motion motors using the PID algorithm (Chang et al., 2009).

In Fig. 1, the UAV (FOXTECH Co. Ltd., Tianjin, China) has 6-axis rotors and maximum takeoff and maximum payload weights of 7 kg and 3 kg, respectively. The maximum flying duration is 15 min on a 12,000 mAh lithium battery supply. The object detection model runs on a Raspberry Pi 4B (6-axis rotors) platform with 2 GB RAM and 32 GB external storage. The Raspberry Pi webcam with 500 megapixels and a 3.4 mm focal length was used as the image capturing component. In addition to the object detection components, the Arduino Mega (Arduino, Scarmagno, Italy) was used to receive the tracking parameters from the Raspberry Pi 4B so as to control the motion motors to track the beehives. The motion motors, which were Herkulex DRS-0601 motors (Dongbu Robot Co., Ltd., Bucheon-Si, South

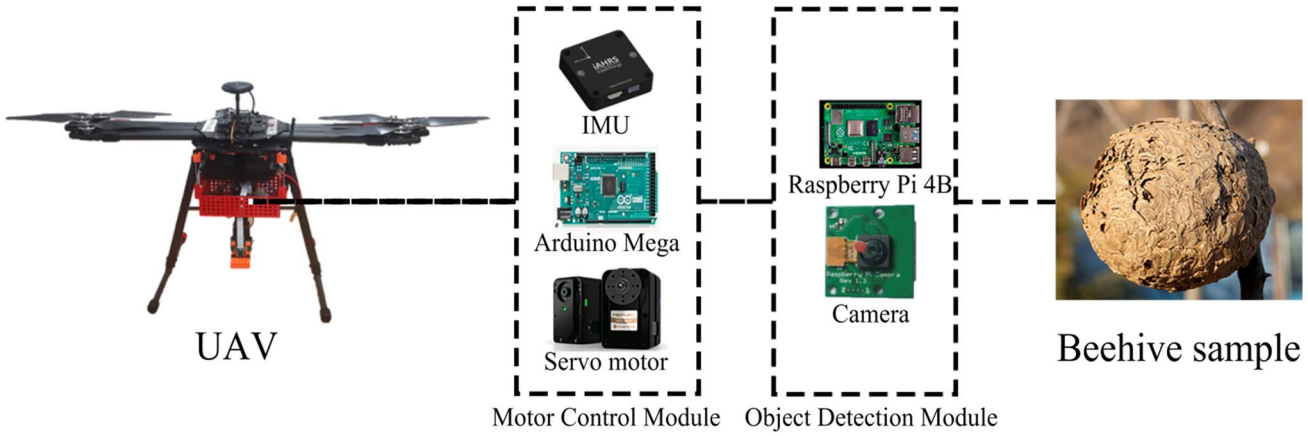


Fig. 1 The entire compositions of dynamic beehive detection and automatic tracking system

Korea), contained two sections installed by an orthogonal connection structure and were used for horizontal and vertical movement. A serial bus was integrated into the motors to achieve 0.163° resolution, and the maximum movement speed was 0.164 s/60° on the 12 V battery supply, which met the requirements of the high-precision tracking system. In addition, the IMU sensors (IAHRS-RB-SDA-V1, ROBOR, South Korea) (Wang et al., 2022a, 2022b) were also fixed on the UAV and the motor to measure the real-time motion attitude of the system and to optimize the motion performance of the automatic tracking system.

Beehive Detection Model

YOLO is a classical single-stage object detection model that offers good detection accuracy and fast inference speed (Hu et al., 2021). YOLO V5 (Yan et al., 2021) is the latest version, and it is more accurate, rapid, and uses fewer parameters than the former versions. YOLO V5s (Wang & He, 2021) is the lightest version of YOLO V5, and its network structure is shown in Fig. 2 (Zhang & Li, 2022). The YOLO V5s consists of Backbone, Neck, and Head modules. The Backbone module is designed to extract special features of the image and contains Focus, C3, and SPP network structure. Focus is used to slice and downsample the input images to reduce computation and accelerate the network training speed. The C3 layer is composed of a bottleneck and three convolution structures that accelerate the network inference speed. The SPP module is used to connect the input feature map to the pooling kernel of sizes 5, 9, and 13 for maximum pooling and then fuse the features at four different scales to strengthen the multi-scale expression ability of the features and improve the receptive field of the model. The Head module is used for network prediction to output the category of the prediction box, confidence degree, and object offset. The YOLO V5 model requires high-performance GPU or CPU platforms to perform well (Hurtik et al., 2022), although it works slowly on edge computing devices such as Raspberry Pi. Therefore, it is necessary to simplify the YOLO V5 model to give it a lighter weight to run on Raspberry Pi to establish and deploy the beehive detection system.

Shuffle Block is a lightweight computing module proposed by ShuffleNet V2 (Ma et al., 2018) based on ShuffleNet V1 and MobileNet V2, which have equal input–output channels and pruned convolutional layers, and decreases the branch structures and basic element operations. Thus, the network inference is accelerated. According to the YOLO V5-Lite model proposed by Chen and Gong (2021), the convolution layer and C3 layer of Backbone belonging to YOLO V5 are replaced with the Shuffle Block module so that the number of original model parameters is fewer, and the network convergence is accelerated. At the same time, considering that the convolutional layer is designed to extract the spatial and channel features, deep detachable convolution is introduced into the Head layer of YOLO V5, and the C3 layer is replaced by deep detachable convolution. In theory, the parameters and computing would be reduced, and the model's accuracy would not be significantly affected (Yang et al., 2021).

After the YOLO V5s model was simplified, 2500 original images containing beehive objects were prepared, mainly from Internet downloading and manual hand-held camera shooting to train the model. Meanwhile, the OpenCV tool was used to enhance the original image dataset using rotation, flipping, brightness adjustment, and other methods. The total number of original images was expanded to 10,000, four times that of the original dataset. After that, Labeling software was used to label the image dataset and establish the beehive label. Finally, the dataset was divided into a ratio of 7:3 for the training and testing datasets to evaluate the model performance (Meister et al., 2021).

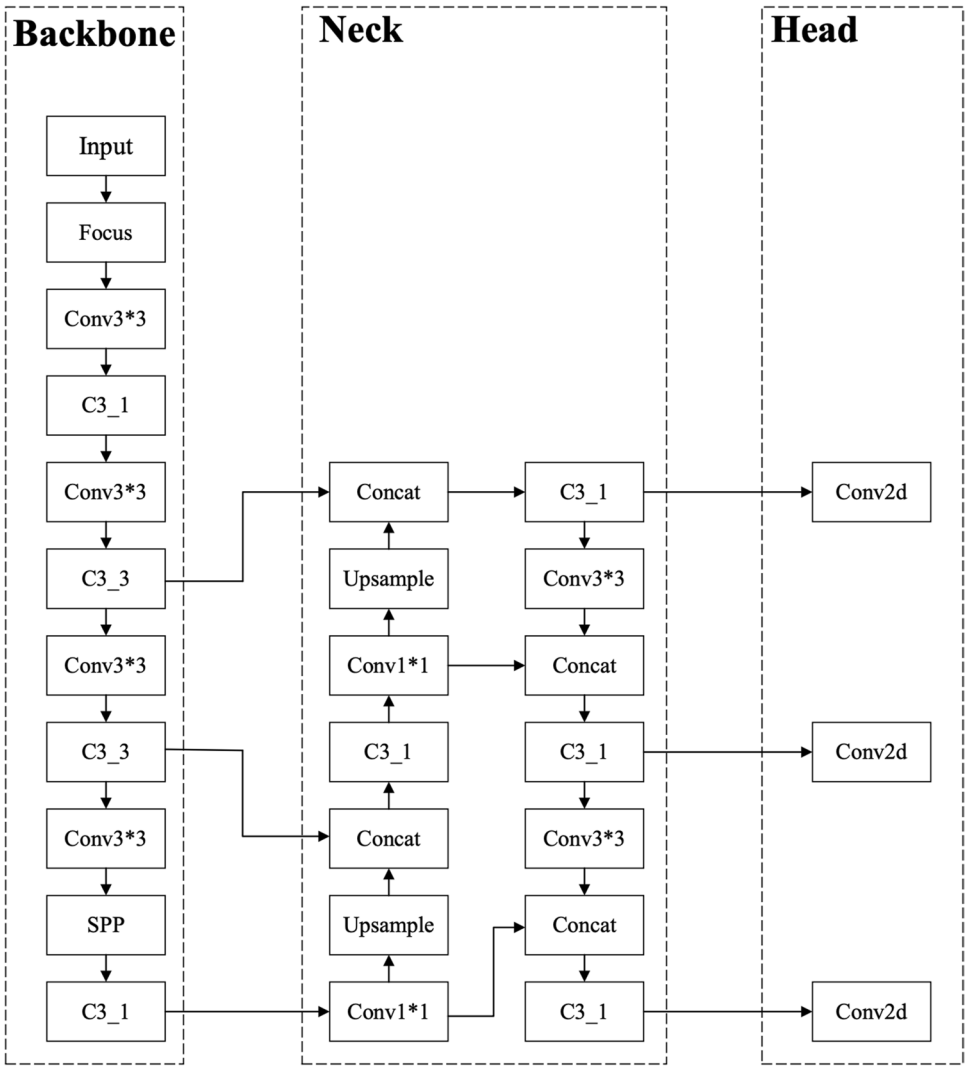


Fig. 2 Overview of network structure about YOLO V5s

Beehive Automatic Tracking System

Traditional destruction of beehives relies on a manual operation by humans, while the beehive detection and tracking system can make it automatic, reducing potential human damage. When the system detected the beehive, the offset data for the beehive center point was calculated. The key to adjusting the motion motors based on the offset data was the PID algorithm (Ma et al., 2022), which was used as a control method by the tracking system. The steps for generating control parameters are as follows:

(1) Calculating the observed difference:

$$diff = -\frac{K_d * (fb[n] - fb[n-1])}{t} \quad (1)$$

where K_d is the difference coefficient, $fb[n]$ represents the input observation for the n th value of the PID algorithm, t denotes the time since the last PID update, and $*$ means multiplication arithmetic.

(2) Calculating the error:

$$err = exp - fb[n] \quad (2)$$

where exp represents the expected value of PID, which is the center coordinator of camera vision.

(3) Calculating the error integral:

$$int[n] = int[n-1] + K_i * err * t \quad (3)$$

where $int[n]$ represents the n th integral of PID and K_i represents the integral coefficient.

(4) Calculating the control output of PID:

$$u[n] = u[n-1] + \left(K_p * err + diff + int[n]\right) \quad (4)$$

where $u[n]$ denotes the n th output of PID, and K_p is the ratio coefficient. The equations were applied on Raspberry Pi 4B to calculate tracking data.

Experimental Methods

Description of Experimental Site

The experiments in this study were of two types: object detection performance and object automatic tracking system performance experiments. The experimental site was located at the College of Agriculture and Life Sciences, Kangwon National University (37°52′10″ N, 127°44′32″ E). The object detection performance experiment included the model performance analysis and the dynamic change of object detection accuracy with distance, as shown in Fig. 3. The performance experiment for the automatic object tracking system included the tracking consumption time and the variation characteristic of the rotation angle of the tracking system during UAV movement.

Description of Object Detection Model Training and Testing Environment

The environment of the object detection model in this paper included the training and testing environment. The hardware and software environments are shown in Table 1. Before the model was trained, the images input into the detection model were normalized, the dataset images were scaled to 320×320, the epoch parameter was set to 200, and the batch size was set to 32. When the model training was completed, the testing dataset was used to evaluate model performance, where the confidence threshold of the test set was 0.001, and the intersection over union (IoU) was set to 0.6 (Wang et al., 2022a, 2022b). In the process of image prediction, the results were treated with non-maximum suppression (Shi et al., 2019), the threshold was set to 0.45, and the confidence threshold was set to 0.25 (Song et al., 2019).

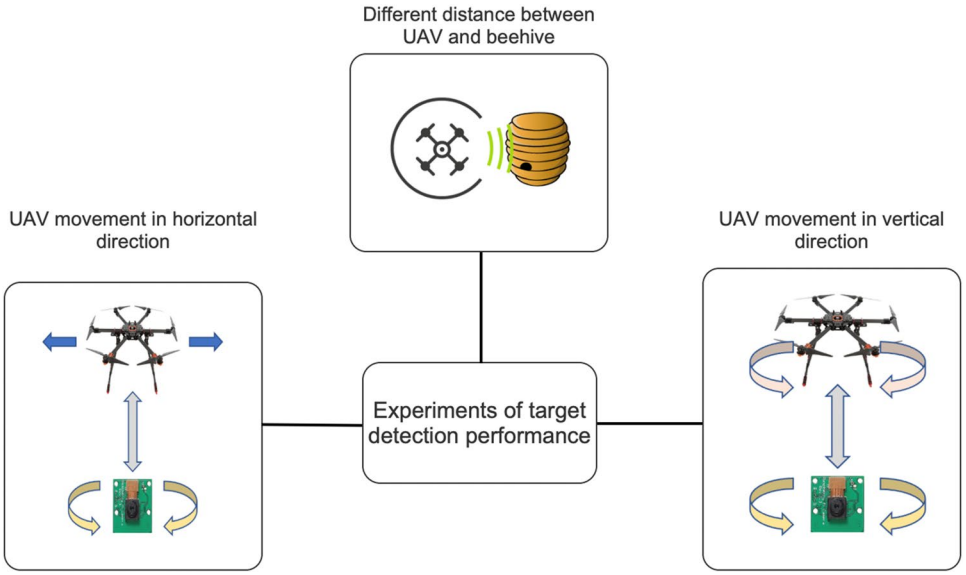


Fig. 3 Schematic diagram of object detection and tracking performance experiments

Table 1 Running environment parameters for the detection model

Category	Training environment	Testing environment	Running environment
CPU	Inter® Xeon® Gold 6240 @2.59 GHz	i5-10,500@2.59 GHz	Broadcom BCM2711@1.5 GHz
GPU	NVIDIA GTX 3090@24 GB	NVIDIA GTX970@6 GB	None
RAM	128 GB	32 GB	4 GB
PyCharm version	2021.3.2	2021.3.2	2021.3.2
Python version	3.8.10	3.8.10	3.8.10
PyTorch version	1.8.0	1.8.0	1.8.0
CUDA version	11.1	11.1	11.1
cuDNN version	8.0.5	8.0.5	8.0.5

Criteria for Model Performance

In the experiments, the model parameters, FLOPs, mean average precision (mAP), precision, recall, and F1_score were used to represent the object detection performance (Nepal & Eslamiat, 2022). The equations are as follows:

$$mAP = \sum_{q=1}^{Q} \frac{AveP(q)}{Q} \tag{5}$$

$$F1_score = 2 * \frac{(precision * recall)}{precision + recall} \tag{6}$$

$$Precision = \frac{TruePositive}{TruePositive + FalsePositive} \tag{7}$$

$$Recall = \frac{TruePositive}{TruePositive + FalseNegative} \tag{8}$$

where mAP is calculated by taking the mean of the average precision of all the classes, q is the number of queries, and $AveP(q)$ is the AP. The IoU are 0.5 and 0.5–0.95, respectively; thus, the criteria of mAP are mAP@0.5 and mAP@0.5–0.95, respectively.

In addition to the object tracking system, the standard deviation (STD) is used to denote the stability of the rotation angle of the UAV and the object tracking system. The equation is:

$$STD = \sqrt{\frac{1}{N} \sum_{i=1}^{N} \left(x_i - \bar{x}\right)^2} \tag{9}$$

where x_i and $\bar{x}$ represent the rotation angle and mean rotation angle during the UAV rotation process in degrees, and N represents the number of samples. On the basis of the above criteria, the experiments were implemented to obtain the testing data and results.

Results and Discussion

Model Performance on Object Detection

Comparison of Object Detection Between YOLO V5s and Simplified YOLO V5-Lite

The established automatic beehive tracking system is based on accurate object detection, meaning that the model's performance is crucial to the system. The model performance is shown in Table 2, and the beehive detection images are shown in Fig. 4 based on the running environment referred to in "Description of Object Detection Model Training and Testing Environment" section. As can be seen in Table 2, the params (the number of model parameters) for the simplified YOLO V5-Lite decreased from 7.2 million to 0.78 million, and the FLOPs decreased from 4.0 Giga to 0.7 Giga, indicating that the computational load of the model has been significantly reduced. The testing accuracy of YOLO V5-Lite is 0.983, which is 0.012 larger than that of the YOLO V5s. And the recall result is also improved by 3.66%. The F1_score of YOLO V5-Lite is 0.984, whereas that of YOLO V5s is 0.995. The mAP@0.5 of the YOLO V5-Lite is only 0.001 larger than that of YOLO V5s. The mAP@0.5–0.95 is 0.877, which indicates that the simplified model can still efficiently obtain information about the object frame in the image compared with the YOLO V5s, which means that the regression classification accuracy of the object frame marked by YOLO V5-Lite model is not negatively influenced. However, considering the single distribution feature of beehives, that is, beehives usually exist independently in a particular place, it is enough for the rapid detection of beehive objects with the proposed model (Xue et al., 2022). The YOLO V5-Lite was deployed in the testing environment to observe the actual beehive image inference performance. The frame rate reached 38.5 FPS, whereas the FPS of YOLO V5s was only 20, which increased by 92.5% than the YOLO V5s. Therefore, the inference speed improved, meaning the model can perform well on an edge device. The beehive detection results in Fig. 4 show that the system can detect objects and perform well, and the size of the object box influences the detection effect. Thus, the detection changed distance with model accuracy test should be implemented to evaluate the practical result.

Table 2 Comparison results for the YOLO V5s and YOLO V5-Lite models

Criteria	YOLO V5s	YOLO v5-Lite
params	7.2 M	0.78 M
FLOPs	4.0G	0.7G
Precision	0.971	0.983
Recall	0.929	0.963
F1_score	0.995	0.984
mAP@0.5	0.994	0.995
mAP@0.5–0.95	0.875	0.877
FPS (testing environment)	20	38.5
FPS (running environment)	0.5	3.15

Dynamic Variation in Object Detection Accuracy with Distance

In the dynamic distance change experiment, a 500-megapixel camera attached to the Raspberry Pi 4B was used to



Fig. 4 Detection results of beehives

detect the beehive in real time. The distance between the object and the camera was changed continually to obtain the corresponding data for object detection accuracy and distance, and the results are shown in Fig. 5, which shows that the detection accuracy tends to decrease gradually with the increase in distance. When the distance was 1.2 m, the detection accuracy reached 96%, and the real-time mean FPS was 3.15. When the distance increased to 1.5 m, the detection accuracy decreased to 85%. The main reason for this is that the regression and classification performance decreased with the size of the object box for the 1.5 m distance. When the distance was maintained at 1.5–2.1 m, the

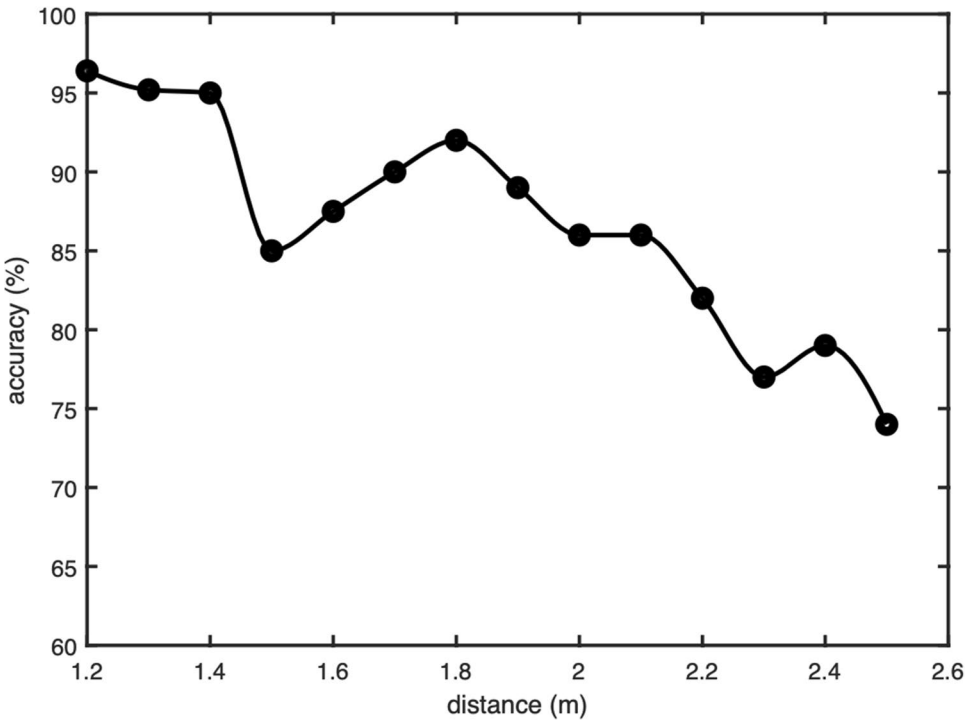


Fig. 5 Results for dynamic detection accuracy between the changed distance of camera and beehive

detection accuracy first increased and then decreased, indicating that the detection effect of YOLO V5-Lite was related to the parameters and network structure, and the size of the object frame. This result provides important data for setting the suitable detection distance for beehive governance. When the distance increased to 2.5 m, the detection accuracy decreased to 74%. When the distance was larger than 2.5 m, the frame could not be generated, meaning that the model could not work with such a small object box. Therefore, a reasonable distance is significant for beehive detection, tracking, and governance.

Elapsed Time for Beehive Tracking

Tracking speed is one of the key indexes of the system. After the detection of a beehive, the object center relative offset pixel was calculated to be transmitted to the Arduino Mega (Adel et al., 2018) platform to control the motion motors. The PID algorithm was used as a feedback control method to adjust the motion parameters of the horizontal and vertical motors, respectively. The tracking time is shown in Fig. 6. The results in Fig. 6 show that the tracking system completed 90% of the motor adjustment process. Considering the characteristic of the PID algorithm, the entire system becomes stable gradually after several vibrations of servo motors. As can be seen from Fig. 6, the total elapsed time needed to complete the tracking process is about 10 s, making the beehive close to the center point. When the system tends to be stable, pixel offset still occurs. The reason for this is that YOLO V5-Lite is a single-stage object detection model, and it obtains classification results based on the regression of the object in the object box. Therefore, the detection represents probabilistic results rather than precise results. Although the tracking system shows the phenomenon of oscillation in the steady state, it is enough to complete the effective tracking of the beehive, considering the beehive diameter is usually 10–50 cm.

Yaw Testing Results for the UAV and Tracking System

In the actual process of beehive management, it is necessary to adjust the UAV to help find objects. Therefore, it is of great significance to evaluate the object tracking performance of the system by recording the yaw angle variation of the UAV and the tracking system, respectively. Yaw angle was measured using the installed IMU sensor. In the experiment, the total movement distance of the UAV was 30 cm with left and right rotation directions. The results are shown in Fig. 7. As can be seen from Fig. 7, when the UAV executed a left rotation, the tracking system adjusted the direction of the camera automatically along with the UAV. Then, with the instruction of the PID algorithm, the system began to track the beehive and finally succeeded in positioning the beehive to the center point of the image. When the UAV turned in the other direction, the tracking system adjusted its orientation and relocated the beehive. In the experiment, the maximum yaw of the tracking system was 19.6°, and the minimum was – 19.3°, respectively. The reason for this was

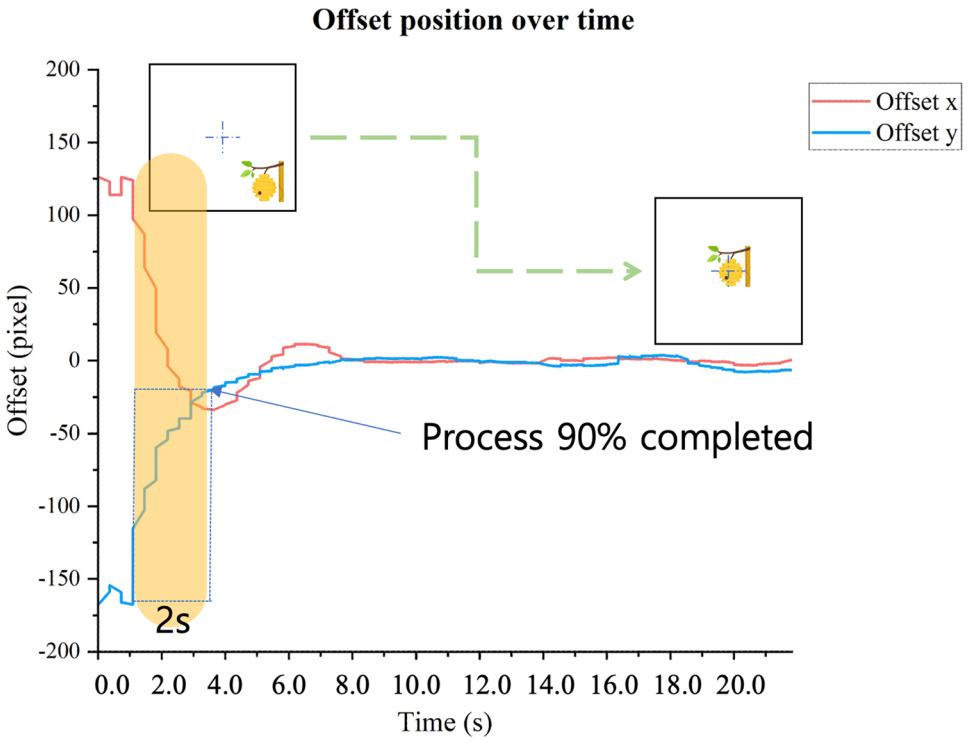


Fig. 6 Experiment results of tracking consumption time

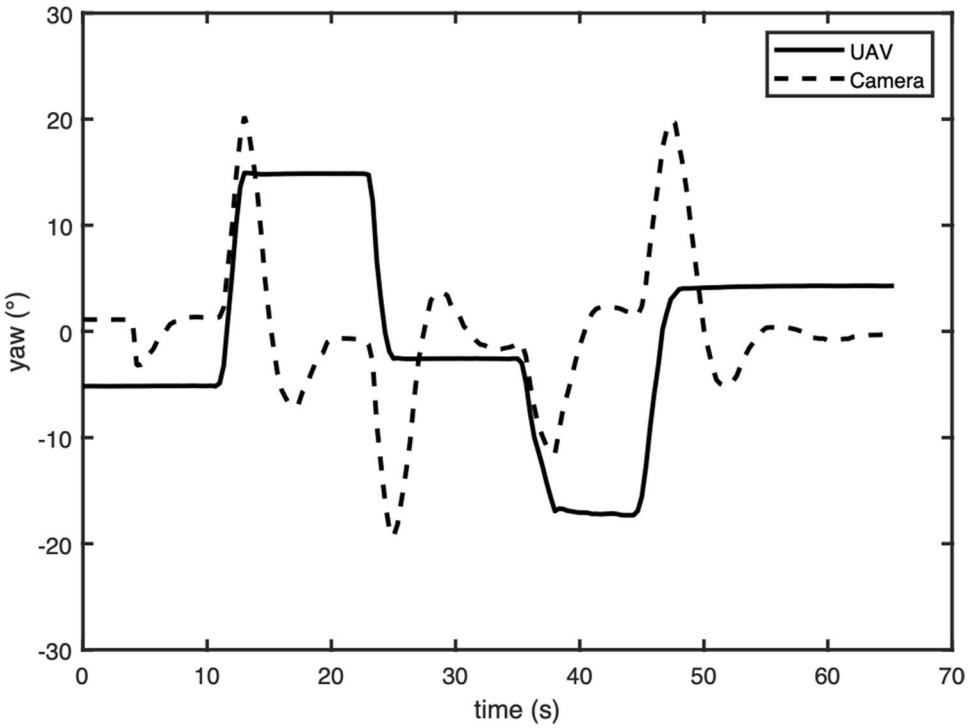


Fig. 7 Yaw variation results for the UAV and tracking system

that the PID algorithm overshot in the process of adjusting the tracking motors, which accelerated the convergence of the tracking system. On the other hand, the overall STD of the UAV was 9.30°, whereas the object tracking system's overall STD was only 6.54°, which indicates that despite the wide variation in the object tracking system, the yaw fluctuations could be reduced by the system, which helps the system to lock the beehive effectively and steadily.

Discussion

The simplified YOLO V5-Lite in this study significantly reduced the number of model parameters and computational complexity, and it could be deployed easily on an edge platform. The results of the performance analysis showed that the inference speed improved by 92.5%. Although the F1_ score accuracy of the model was slightly lower than that of the YOLO V5s, the model can be deployed on a Raspberry Pi 4B to achieve real-time beehive detection. Raspberry Pi 4B was able to detect 96% of dynamic objects when the beehive was 1.2 m away from the camera, suggesting that Raspberry Pi 4B's slower reasoning speed did not significantly affect its inference accuracy. With the continuous expansion of the distance between the honeycomb and the camera, the inference accuracy decreased and then increased until the beehive was undetectable. Although the maximum distance to detect the beehive was 2.5 m, considering the limited shooting range of the flamethrower, the detection distance is enough in most cases. For the actual volume of a normal beehive, the threshold of inference confidence was set to 70% in this study. Saffari et al. (2021) used Raspberry Pi 4B and YOLO V5s to conduct experimental research on object detection at 10 FPS with an input image size of 120 × 120. In terms of beehive governance, a large resolution, such as 320 × 320, and a long distance are necessary. The camera focal length used in this study was 3.4 mm; thus, a future study could use a camera with a larger focal length with a smaller input image size to achieve better precision and a more rapid inference speed.

Although UAV and flamethrowers are often applied in destroying beehives, the UAV could not fly completely smoothly, and the flamethrowers could also not focus on beehives automatically, while the key to destroying beehives precisely is accurate aiming at the beehive. With the beehive detection system, the PID algorithm and dual servos were used to track the beehive object automatically according to the model inference results. The system completed 90% of the tracking process within 2 s. Because the object frame size that was output by the object detection model is the probability result obtained by regression classification, the final object pixel offset also changes dynamically. That is the main reason for the PID algorithm. Considering the real size of beehives, the error does not have a significant influence. The test results showed that the tracking system followed the object and UAV automatically without manual operation. Hence, the system was used to manage beehives and reduce secondary damage to humans. The Raspberry Pi 4B limited the system's performance; therefore, a better edge

computing platform, such as the Intel NUC mini-computer (Intel Co., Santa Clara, CA, USA), would obtain a better inference speed and rapid tracking performance. On the other hand, a high-performance platform would require more power. Thus, a future study should balance the performance, power supply, image quality, and UAV mounting ability to build a better tracking system.

Conclusions

This study proposed an improved beehive detection model based on YOLO V5s according to beehive governance requirements. A beehive tracking system was established based on object detection. The test results showed that the YOLO V5-Lite could reduce parameters with only a small accuracy loss. The inference speed increased by 92.5%; thus, the model is easy to build on an edge platform. With the PID algorithm and dual motion motors, the beehive tracking system completed 90% of the tracking process within 2 s. When the UAV moved, the tracking system followed the object automatically. The established system could remove wild beehives and reduce secondary damage to rescue workers. Considering that Raspberry Pi 4B has inference limitations, a better platform should be introduced for beehive detection, while the better platform also brings new challenges to the comprehensive balance of the whole system.

Acknowledgements The authors would like to thank all authors for openly providing the source codes used in the experimental comparison in this work.

Funding This research was funded by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. NRF-2021R1F1A1055992). It was also partly supported by the Guangdong Provincial organization of China and the implementation project in 2021 (YUECAINONG [2021] No. 37) "Integration and Demonstration of Key Technology Models of Modern Agriculture in Guangdong Province."

Declarations

Conflict of Interest The authors declare no competing interests.

References

Adel, Z., Hamou, A. A., & Abdellatif, S. (2018). Design of real-time PID tracking controller using Arduino Mega 2560for a permanent magnet DC motor under real disturbances, in: *2018 international conference on electrical sciences and technologies in Maghreb (CISTEM)*, *Presented at the 2018 international conference on electrical sciences and technologies in Maghreb (CISTEM),* pp. 1–5. https://doi.org/10.1109/CISTEM.2018.8613560

Brugnerotto P., Seraglio S., Schulz M., Gonzaga L., Fett R., & Costa A. (2021). Pyrrolizidine alkaloids and beehive products: A review. *Food Chemistry, 342*:128384. https://doi.org/10.1016/j.foodchem.2020.128384

Chang, H. J., Kim, P. J., Song, D. S., & Choi, J. Y. (2009). Optical image stabilizing system using multirate fuzzy PID controller for mobile device camera. *IEEE Transactions on Consumer Electronics, 55*, 303–311. https://doi.org/10.1109/TCE.2009.5174385

Chen, X., & Gong, Z. (2021). YOLOv 5-Lite: Lighter, faster and easier to deploy. Available online: https://pythonawesome.com/yolov5-lite-lighter-faster-and-easier-to-deploy.

Dainelli, R., Toscano, P., Di Gennaro, S. F., & Matese, A. (2021). Recent advances in unmanned aerial vehicle forest remote sensing—A systematic review. Part i: A General Framework. *Forests, 12*, 327. https://doi.org/10.3390/f12030327

Gunes, H., & Gungormus, A. (2022). Identification of honey bee (Apis mellifera) larvae in the hive with faster R-CNN for royal jelly production. *Journal of Apicultural Research, 61*, 338–345. https://doi.org/10.1080/00218839.2022.2030023

Hu, X., Liu, Y., Zhao, Z., Liu, J., Yang, X., Sun, C., Chen, S., Li, B., & Zhou, C. (2021). Real-time detection of uneaten feed pellets in underwater images for aquaculture using an improved YOLO-V4 network. *Computers and Electronics in Agriculture, 185*, 106135. https://doi.org/10.1016/j.compag.2021.106135

Huang, R., Pedoeem, J., & Chen, C. (2018). YOLO-LITE: A real-time object detection algorithm optimized for Non-GPU computers, in: *2018 IEEE international conference on big data (Big Data). Presented at the 2018 IEEE international conference on big data (Big Data),* pp. 2503–2510. https://doi.org/10.1109/BigData.2018.8621865

Hurtik, P., Molek, V., Hula, J., Vajgl, M., Vlasanek, P., & Nejezchleba, T. (2022). Poly-YOLO: Higher speed, more precise detection and instance segmentation for YOLOv3. *Neural Computing and Applications, 34*, 8275–8290. https://doi.org/10.1007/s00521-021-05978-9

Legaspi, K.R.B., Sison, N.W.S., & Villaverde, J.F. (2021). Detection and classification of whiteflies and fruit flies using YOLO, in: *2021 13th international conference on computer and automation engineering (ICCAE). Presented at the 2021 13th international conference on computer and automation engineering (ICCAE),* pp. 1–4. https://doi.org/10.1109/ICCAE51876.2021.9426129

Ma, R., Li, S., Zhang, B., & Li, Z. (2022). Towards fast and robust real image denoising with attentive neural network and PID controller. *IEEE Transactions on Multimedia, 24*, 2366–2377. https://doi.org/10.1109/TMM.2021.3079697

Ma, N., Zhang, X., Zheng, H.-T., & Sun, J. (2018). ShuffleNet V2: Practical guidelines for efficient CNN architecture design, in: *Ferrari, V., Hebert, M., Sminchisescu, C., Weiss, Y. (Eds.), Computer vision – ECCV 2018, lecture notes in computer science. Springer International Publishing, Cham,* pp. 122–138. https://doi.org/10.1007/978-3-030-01264-9_8

Meister, S., Wermes, M. A. M., Stüve, J., & Groves, R. M. (2021). Review of image segmentation techniques for layup defect detection in the automated fiber placement process. *Journal of Intelligent Manufacturing, 32*, 2099–2119. https://doi.org/10.1007/s10845-021-01774-3

Nader, R. A., Mackieh, R., Wehbe, R., Obeid, E. D., Sabatier, J. M., & Fajloun, Z. (2021). Beehive products as antibacterial agents: A review. *Antibiotics, 10*, 717. https://doi.org/10.3390/antibiotics10060717

Nepal, U., & Eslamiat, H. (2022). Comparing YOLOv3, YOLOv4 and YOLOv5 for autonomous landing spot detection in faulty UAVs. *Sensors, 22*, 464. https://doi.org/10.3390/s22020464

Rahman, M. F. F., Fan, S., Zhang, Y., & Chen, L. (2021). A comparative study on application of unmanned aerial vehicle systems in agriculture. *Agriculture, 11*, 22. https://doi.org/10.3390/agriculture11010022

Saffari, A., Tan, S.Y., Katanbaf, M., Saha, H., Smith, J.R., & Sarkar, S. (2021). Battery-free camera occupancy detection system, in: *Proceedings of the 5th international workshop on embedded and mobile deep learning. Presented at the MobiSys '21: The 19th annual international conference on mobile systems, applications,*

and services, ACM, Virtual WI USA, pp. 13–18. https://doi.org/10.1145/3469116.3470013

Shi, M., Ouyang, P., Yin, S., Liu, L., & Wei, S. (2019). A fast and power-efficient hardware architecture for non-maximum suppression. *IEEE Transactions on Circuits and Systems II: Express Briefs, 66*, 1870–1874. https://doi.org/10.1109/TCSII.2019.2893527

Song, Y., Pan, Q.-K., Gao, L., & Zhang, B. (2019). Improved non-maximum suppression for object detection using harmony search algorithm. *Applied Soft Computing, 81*, 105478. https://doi.org/10.1016/j.asoc.2019.05.005

Tian, Y., Yang, G., Wang, Z., Wang, H., Li, E., & Liang, Z. (2019). Apple detection during different growth stages in orchards using the improved YOLO-V3 model. *Computers and Electronics in Agriculture, 157*, 417–426. https://doi.org/10.1016/j.compag.2019.01.012

Wahyutama, A. B., & Hwang, M. (2022). YOLO-based object detection for separate collection of recyclables and capacity monitoring of trash bins. *Electronics, 11*, 1323. https://doi.org/10.3390/electronics11091323

Wang, D., & He, D. (2021). Channel pruned YOLO V5s-based deep learning approach for rapid and accurate apple fruitlet detection before fruit thinning. *Biosystems Engineering, 210*, 271–281. https://doi.org/10.1016/j.biosystemseng.2021.08.015

Wang, X., & Liu, J. (2021). Tomato anomalies detection in greenhouse scenarios based on YOLO-Dense. *Frontiers in Plant Science, 12*, 634103. https://doi.org/10.3389/fpls.2021.634103

Wang, P., Han, X., Son, Y., Mo, C., Kim, D., Kim, S., Kim, J., & Chung, W. (2022a). Development and analysis of multi-functional beekeeping loading box based on electric tracked transport vehicle. *Journal of Biosystems Engineering, 47*, 13–27. https://doi.org/10.1007/s42853-021-00125-7

Wang, Z., Jin, L., Wang, S., & Xu, H. (2022b). Apple stem/calyx real-time recognition using YOLO-v5 algorithm for fruit automatic loading system. *Postharvest Biology and Technology, 185*, 111808. https://doi.org/10.1016/j.postharvbio.2021.111808

Wu, J., Peng, L., Li, J., Zhou, X., Zhong, J., Wang, C., & Sun, J. (2021). Rapid safety monitoring and analysis of foundation pit construction using unmanned aerial vehicle images. *Automation in Construction, 128*, 103706. https://doi.org/10.1016/j.autcon.2021.103706

Xue, J., Zheng, Y., Dong-Ye, C., Wang, P., & Yasir, M. (2022). Improved YOLOv5 network method for remote sensing image-based ground objects recognition. *Soft Computing*. https://doi.org/10.1007/s00500-022-07106-8

Yan, B., Fan, P., Lei, X., Liu, Z., & Yang, F. (2021). A real-time apple targets detection method for picking robot based on improved YOLOv5. *Remote Sensing, 13*, 1619. https://doi.org/10.3390/rs13091619

Yang, R., Lu, X., Huang, J., Zhou, J., Jiao, J., Liu, Y., Liu, F., Su, B., & Gu, P. (2021). A multi-source data fusion decision-making method for disease and pest detection of grape foliage based on ShuffleNet V2. *Remote Sensing, 13*, 5102. https://doi.org/10.3390/rs13245102

Yu, S. (2021). Thoughts on the disposal action and effective control of wasps. *Fire community, 7*, 31–3235. https://doi.org/10.16859/j.cnki.cn12-9204/tu.2021.22.013

Zhang, P., & Li, D. (2022). EPSA-YOLO-V5s: A novel method for detecting the survival rate of rapeseed in a plant factory based on multiple guarantee mechanisms. *Computers and Electronics in Agriculture, 193*, 106714. https://doi.org/10.1016/j.compag.2022.106714

Zhuang, J., Xin, Huang, & Ye, X. (2022). *Bee colony flow monitoring system based on SSD algorithm* (pp. 1–3). ICETIS 2022; 7th international conference on electronic technology and information science.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Peng Gao[1] · Kangbeen Lee[2,3] · Lukas Wiku Kuswidiyanto[2,3] · Seung-Hwa Yu[4] · Kai Hu[1] · Gaotian Liang[1] · Yufeng Chen[1] · Weixing Wang[1] · Fei Liao[1] · Yu Seok Jeong[4] · Moon-Seok Jeon[4] · Inchan Choi[4] · Xiongzhe Han[2,3]

Peng Gao
gaopeng.peng@scau.edu.cn

Kangbeen Lee
kangtande@gmail.com

Lukas Wiku Kuswidiyanto
lukaswiku@kangwon.ac.kr

Seung-Hwa Yu
yush6210@korea.kr

Kai Hu
hukai2020@stu.scau.edu.cn

Gaotian Liang
201621190216@stu.scau.edu.cn

Yufeng Chen
chenyufeng@stu.scau.edu.cn

Weixing Wang
weixing@scau.edu.cn

Fei Liao
upupfei@stu.scau.edu.cn

Yu Seok Jeong
dbtjr2665@naver.com

Moon-Seok Jeon
sd118687@naver.com

Inchan Choi
inchchoi@korea.kr

[1] College of Electronic Engineering, South China Agricultural University, Guangzhou 510642, China

[2] Department of Biosystems Engineering, College of Agriculture and Life Sciences, Kangwon National University, Chuncheon 24341, Korea

[3] Interdisciplinary Program in Smart Agriculture, College of Agriculture and Life Sciences, Kangwon National University, Chuncheon 24341, Korea

[4] Department of Agricultural Engineering, Rural Development Administration, National Institute of Agricultural Sciences, Jeonju 54875, Korea